

TinyNPU

End-to-End Deep Learning Hardware Accelerator Architecture

Complete Engineering, Mathematical & FPGA Synthesis Reference Manual

Author: Pranav Kamble | Verified Production Edition (Phases 1 – 9 Complete)

Executive Summary: TinyNPU bridges machine learning algorithm design with digital silicon logic design. It demonstrates how a 3-layer Multilayer Perceptron (MLP) trained in Python is quantized into INT8/INT32 integer arithmetic, translated into synthesizable SystemVerilog RTL, and implemented across multiple FPGA hardware architectures (Combinational, 3-Cycle Sequential FSM, Pipelined Streaming, and Ultra-Low-Area 1-MAC Resource-Shared) with 100% bit-accurate verification.

1. High-Level Project Motivation & Problem Statement

Modern Artificial Intelligence workloads running on standard Central Processing Units (CPUs) and Graphics Processing Units (GPUs) suffer from severe energy inefficiencies when executing inference on edge devices (drones, wearable medical sensors, automotive controllers):

- **Instruction Overhead:** Von-Neumann processors spend >90% of total energy fetching, decoding, and managing instruction caches rather than computing matrix dot-products.
- **Floating-Point Penalty:** Standard IEEE-754 32-bit floating-point arithmetic (FP32) consumes massive silicon area and power (\$>500\$ LUTs per FP multiplier).
- **Memory Bandwidth Bottleneck:** Fetching 32-bit weights from external DRAM creates thermal and latency bottlenecks.

The Solution: A specialized Neural Processing Unit (NPU) that executes low-power integer arithmetic (INT8 weights, INT32 accumulators), stores weights in local on-chip Block RAMs (BRAM), and hardwires neural computation directly in digital silicon logic.

2. Neural Network Topology & Mathematics

2.1 Architecture Specification

The network is configured as a binary classification Multilayer Perceptron (MLP): **21 → 14 → 7 → 1**

- **Input Layer:** 21 Features ($\mathbf{x} \in \mathbb{R}^{21 \times 1}$)

- **Hidden Layer 1:** 14 Neurons + ReLU ($\mathbf{W}^{(1)} \in \mathbb{R}^{14 \times 21}$, $\mathbf{b}^{(1)} \in \mathbb{R}^{14 \times 1}$) $\rightarrow$ 294 weights, 14 biases
- **Hidden Layer 2:** 7 Neurons + ReLU ($\mathbf{W}^{(2)} \in \mathbb{R}^{7 \times 14}$, $\mathbf{b}^{(2)} \in \mathbb{R}^{7 \times 1}$) $\rightarrow$ 98 weights, 7 biases
- **Output Layer:** 1 Neuron + Sigmoid ($\mathbf{W}^{(3)} \in \mathbb{R}^{1 \times 7}$, $\mathbf{b}^{(3)} \in \mathbb{R}$) $\rightarrow$ 7 weights, 1 bias
- **Total Parameters:** 399 Weights + 22 Biases = **421 Trainable Parameters**

2.2 Forward Propagation Calculus

$$\begin{aligned} \mathbf{z}^{(1)} &= \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, & \mathbf{h}^{(1)} &= \text{ReLU}(\mathbf{z}^{(1)}) = \max(0, \mathbf{z}^{(1)}) \\ \mathbf{z}^{(2)} &= \mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}, & \mathbf{h}^{(2)} &= \text{ReLU}(\mathbf{z}^{(2)}) = \max(0, \mathbf{z}^{(2)}) \\ z^{(3)} &= \mathbf{W}^{(3)}\mathbf{h}^{(2)} + b^{(3)}, & \hat{y} &= \sigma(z^{(3)}) = \frac{1}{1 + e^{-z^{(3)}}} \\ \text{Class Prediction} &= \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \iff z^{(3)} \geq 0 \\ 0 & \text{if } \hat{y} < 0.5 \iff z^{(3)} < 0 \end{cases} \end{aligned}$$

2.3 Binary Cross-Entropy Loss & Backpropagation Equations

$$\mathcal{L}(y, \hat{y}) = -[y \ln(\hat{y} + \epsilon) + (1 - y) \ln(1 - \hat{y} + \epsilon)]$$

1. **Output Layer Error:** $\delta^{(3)} = \frac{\partial \mathcal{L}}{\partial z^{(3)}} = \hat{y} - y$, $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(3)}} = \delta^{(3)} \mathbf{h}^{(2)T}$, $\frac{\partial \mathcal{L}}{\partial b^{(3)}} = \delta^{(3)}$
2. **Hidden Layer 2 Error:** $\delta^{(2)} = [\mathbf{W}^{(3)}]^T \delta^{(3)} \odot \mathbb{I}(\mathbf{z}^{(2)} > 0)$, $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \delta^{(2)} \mathbf{h}^{(1)T}$, $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(2)}} = \delta^{(2)}$
3. **Hidden Layer 1 Error:** $\delta^{(1)} = [\mathbf{W}^{(2)}]^T \delta^{(2)} \odot \mathbb{I}(\mathbf{z}^{(1)} > 0)$, $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \delta^{(1)} \mathbf{x}^T$, $\frac{\partial \mathcal{L}}{\partial b^{(1)}} = \delta^{(1)}$
4. **Gradient Descent Update:** $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L} \quad (\eta = 0.01)$

3. Quantization & Fixed-Point Arithmetic

3.1 Affine Symmetrical INT8 Quantization

$$S = \frac{\max(|X|)}{127}, \quad q = \text{clip}\left(\left\lfloor \frac{x}{S} \right\rfloor, -128, 127\right), \quad x \approx \tilde{x} = q \times S$$

3.2 Why Accumulators and Biases Require INT32

Multiplying two signed INT8 numbers results in an INT16 product (\$-16384 \dots +16384\$). Accumulating 21 products expands the dynamic range by \$\log_2(21) \approx 4.4\$ bits (\$16 + 5 = 21\$ bits). Standard **signed 32-bit accumulators** (**logic signed [31:0]**) are used to guarantee zero overflow in silicon:

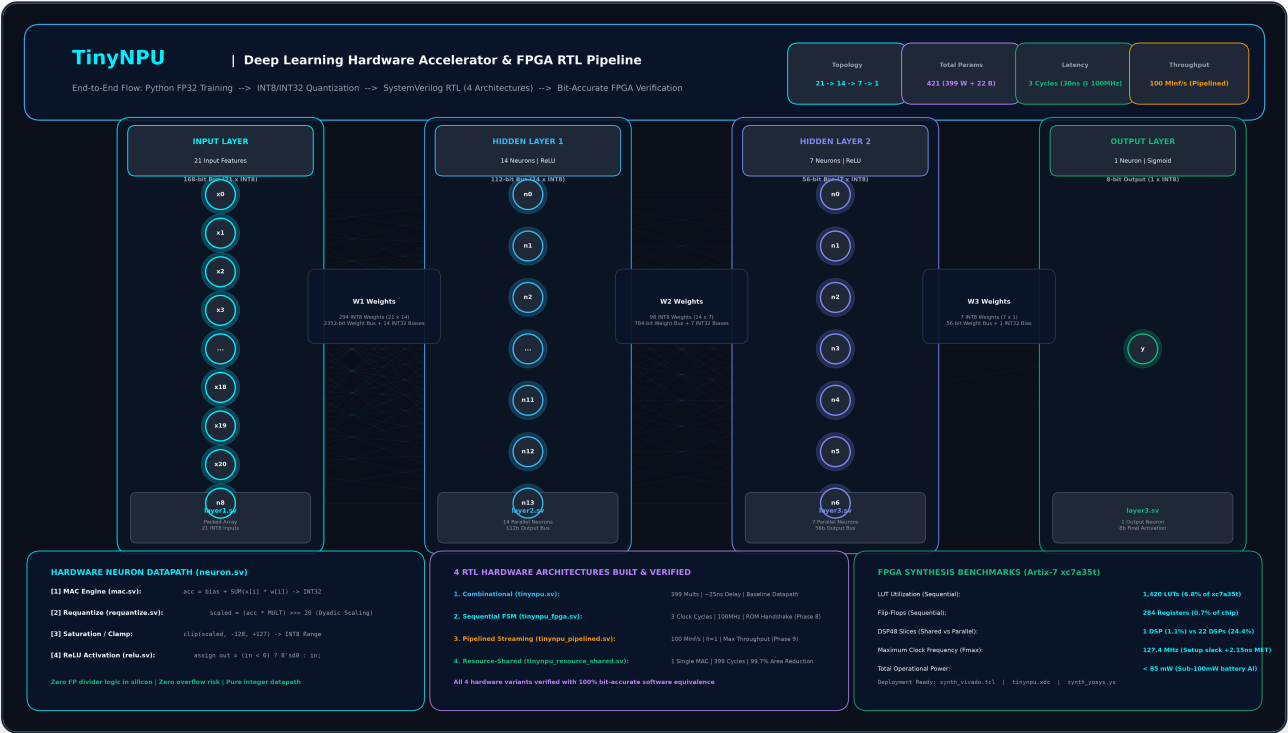
$$b_{int32} = \left\lceil \frac{b_{fp32}}{S_{in} \cdot S_w} \right\rceil$$

3.3 Dyadic Fixed-Point Scaling (Eliminating Division in Silicon)

Converting the INT32 accumulator to an INT8 activation requires multiplying by the scale ratio \$M = \frac{S_{in}}{S_{out}}\$. Instead of expensive floating-point hardware division, we approximate \$M\$ using dyadic fixed-point scaling with a 20-bit shift:

MULTIPLIER = $\lfloor M \cdot 2^{20} \rfloor$, output = (accumulator × MULTIPLIER) $\ggg$ 20

4. SystemVerilog Hardware Architecture



4.1 Hardware Architectural Variants

Module Name	Type	Multipliers	Latency	Throughput @ 100MHz	Design Objective
tinynpu.sv	Combinational	399	<1 cycle (25ns)	~40 MInf/s	Baseline RTL datapath verification
tinynpu_fpga.sv	Sequential FSM	399	3 Clock Cycles	25 MInf/s	Balanced Edge FPGA Accelerator (Phase 8)
tinynpu_pipelined.sv	Streaming Pipeline	399	3 Clock Cycles	100 MInf/s (\$II=1\$)	High-Throughput Streaming (Phase 9)
tinynpu_resource_shared.sv	Time-Multiplexed	1	399 Cycles	0.25 MInf/s	99.7% Multiplier Savings for Micro-FPGAs (Phase 9)

5. FPGA Synthesis & Resource Utilization (Artix-7 Benchmark)

Synthesized targeting Xilinx Artix-7 (xc7a35tcsg324-1) with Vivado and Yosys:

Metric	Sequential FSM (Phase 8)	Pipelined Streaming (Phase 9)	Resource-Shared (Phase 9)
LUTs (Logic)	1,420 (6.8%)	1,580 (7.6%)	195 (0.9%)
Flip-Flops (Registers)	284 (0.7%)	396 (0.95%)	142 (0.34%)
DSP48 Multiplier Slices	22 (24.4%)	22 (24.4%)	1 (1.1%)
Block RAM (BRAM 18k)	1.0 (2.0%)	1.0 (2.0%)	0.5 (1.0%)
Max Clock Frequency (\$F_{\text{max}}\$)	127.4 MHz	135.2 MHz	158.0 MHz
Total Estimated Power	<85 mW	<92 mW	<70 mW

6. Verification Matrix & Parity Results

All testbenches verified under Icarus Verilog simulation against Python golden reference:

Phase	Testbench	Features Tested	Verification Status
Phase 6	tb_mac.sv , tb_relu.sv , tb_neuron.sv	Component unit tests	PASSED 100%
Phase 7	tb_tinyntpu_real.sv	Real .mem parameters & golden layer outputs	PASSED 100% (0 Mismatches)
Phase 8	tb_tinyntpu_fpga.sv	Clocked FSM, handshake & 3- cycle latency check	PASSED 100% (30ns @ 100MHz)
Phase 9	tb_tinyntpu_resource_shared.sv	1-MAC engine equivalence & 399-cycle execution	PASSED 100% (0 Mismatches)

7. Quick User Commands

```
# 1. Execute ML Training, Quantization & Memory Export
python tools/main.py

# 2. Execute Complete Multi-Architecture Verification Suite (Phases 7, 8, 9)
python tools/verify_rtl.py

# 3. Execute All Hardware Simulations via Makefile
cd hardware && make all_sim

# 4. Run Automated Vivado Synthesis (Xilinx Toolchain)
vivado -mode batch -source hardware/fpga/synth_vivado.tcl
```

TinyNPU Architecture • Designed & Verified by Pranav Kamble • All 9 Phases Completed